

MERN-Stack

In Terminal:

- `cd .\backend\`
- `npm init -y`
- `npm install express(@VERSION)` (for instance: [express@4.18.2](#))

create **server.js** :

```
import express from "express"; // package.json --> "type": "module"
const app = express();

app.listen(5001, () =>{
  console.log("Server started on PORT: 5001");
});
```

- **app.listen()** starts the Express server and makes it listen for incoming HTTP requests (the API requests). In this case, it starts on port 5001. (<http://localhost:5001>)
- **Note:** or the import to work, you must change the “type” value to “module” in **package.json**

Go to **package.json** and change these values:

```
...
"type": "module",
"main": "server.js",
"scripts": {
  "dev": "npm server.js", ← This thing is called when we run “npm run dev”
```

Then we can have our first API example in **server.js** for a simple get request.

```
app.get("/api/notes", (req, res) => {
  res.status(200).send("Success"); // or res.send("Success");
});
```

<http://localhost:5001/api/notes> will return the message “Success”.

NOTE: It is better to have **app.listen()** at the bottom for a better readability and to avoid confusion.

There is a problem.

Now changing any part of our code isn’t applied until we Ctrl + C in the terminal and restart the server (npm run dev). To fix that, we install nodemon.

In Terminal:

- `npm install nodemon -D`

And then in package.json we change:

```
"scripts": {
  "dev": "nodemon server.js", ← This thing here is called when we “run npm run dev”
```

Best practice

Instead of having all the `app.get()` methods in the `server.js` file, we can have a cleaner code by making another JavaScript file and keep them there. Then we can access them using `express.Router()`.

server.js:

```
import express from "express"; // package.json --> "type": "module"
import notesRoutes from ".routes/notesRoutes.js";

const app = express();

app.use("/api/notes", notesRoutes); // uses the routes in the file instead of having them all here

app.listen(5001, () =>{
  console.log("Server started on PORT: 5001");
});
```

`/routes/NotesRoutes.js:` (note that `/routes/` is the folder name holding all the routes)

```
import express from "express"

const router = express.Router();

router.get("/", (req, res) => {
  res.status(200).send("You just fetched the notes.");
});

router.post("/", (req, res) => {
  res.status(201).json({message:"Note created successfully!"});
});

router.put("/:id", (req, res) => {
  res.status(201).json({message:"Note created successfully!"});
});

router.put("/:id", (req, res) => {
  res.status(200).json({message:"Note deleted successfully!"});
});

export default router;
```

(We have switched to `Json` for message delivery since it's better.)

We can also go further and move the controllers (all the router.get/router.put/etc.) onto a separate file as separate functions:

- create `./backend/controllers/notesController.js` :

```
export function getAllNotes(req, res){
  res.status(200).send("You just fetched the notes.");
}

export function createNote(req, res){
  res.status(201).json({message:"Note created successfully!"});
}

export function updateNote(req, res){
  res.status(201).json({message:"Note updated successfully!"});
}

export function deleteNote(req, res){
  res.status(200).json({message:"Note deleted successfully!"});
}
```

- change `notesRoutes.js` :

```
import express from "express"
import { createNote, deleteNote, getAllNotes, updateNote } from "../controllers/notesController";

const router = express.Router();

router.get("/", getAllNotes);

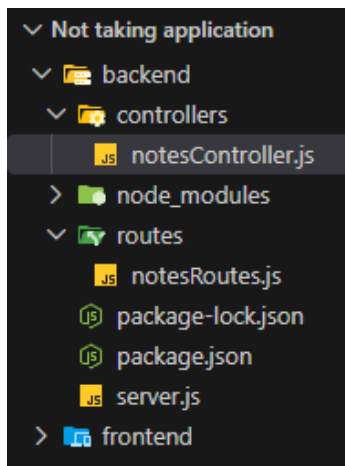
router.post("/", createNote);

router.put("/:id", updateNote);

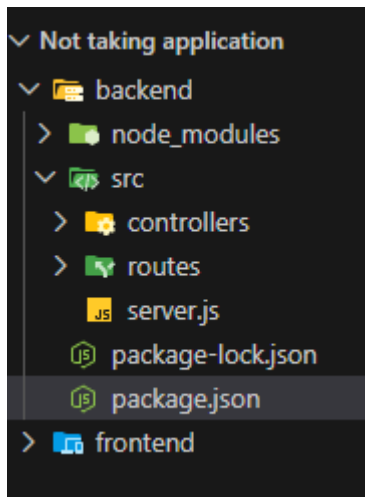
router.put("/:id", deleteNote);

export default router;
```

This is how our hierarchy looks at this point:



And we can also put them in the src folder for a cleaner project:



And change package.json accordingly:

```
"scripts": {
  "dev": "nodemon /src/server.js",
  "start": "node src/server.js" ← This thing is used when the server is implemented and not in
development (ignore for now)
},
```

Installing MongoDB to connect to our database

- run **npm install mongoose**(@VERSION can be used)

note: you can also use `npm install mongodb` but `mongoose` is better.

- Copy the **connection link** during the process of making a cluster on the MongoDB dashboard.

```
mongodb+srv://<user>:<password>@cluster0.34pw2pi.mongodb.net/<db_name>?appName=Cluster0
```

- Use it in a function to initialize a connection.
- For a better readability we make a file under a new folder accordingly: `./config/db.js`

```
import mongoose from "mongoose"

export const connectDB = async () => {
  try{
    await mongoose.connect("PASTE THE CONNECTION LINK HERE");
    console.log("MONGODB CONNECTED SUCCESSFULLY");
  } catch (error){
    console.log("ERROR CONNECTING TO MONGODB", error);
    process.exit(1); // 1: exit with failure, 0: exit with success
  }
}
```

- Then in **server.js** (our main file) add these:

```
import { connectDB } from "./config/db.js";
const app = express(); ← was already there. ignore this.
connectDB();
```

There is a problem

By just pushing this file to github, everyone will have the link to our database by just reading that line! To avoid that:

- Make: `/backend/.env`
- Write:

```
MONGO_URI=mongodb+srv://<user>:<password>@cluster0.34pw2pi.mongodb.net/<db_name>?appName=<clustername>
```

- run **npm install dotenv** in terminal
- Add this to **server.js**:

```
import dotenv from "dotenv";
dotenv.config();
```

- Then change the **db.js** file accordingly:

```
import mongoose from "mongoose"
export const connectDB = async () => {
  try{
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MONGODB CONNECTED SUCCESSFULLY");
  } catch (error){
    console.log("ERROR CONNECTING TO MONGODB", error);
    process.exit(1); // exit with failure
  }
}
```

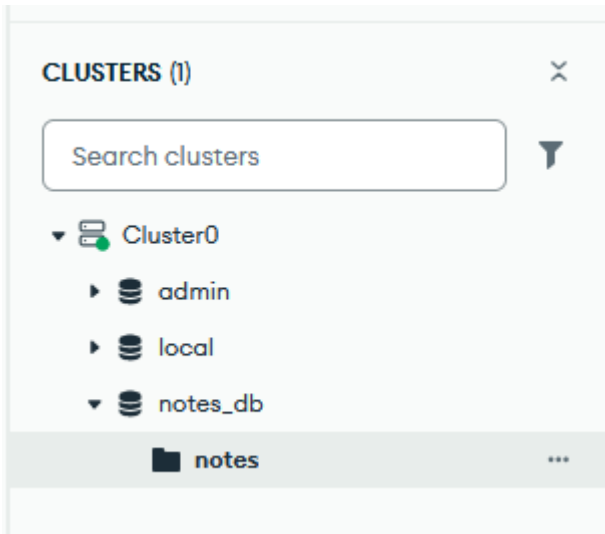
- Then we make sure our env file is hidden and invisible in our github repository. (using **gitignore**)

Making a model for notes

- **/src/models/Note.js** :

```
import mongoose from "mongoose";
// 1st step: create a schema
// 2nd step: create a model based off of that schema
const noteSchema = new mongoose.Schema(
  {
    title:{
      type: String,
      required: true
    },
    content:{
      type: String,
      required: true
    }
  },
  { timestamps: true } // createdAt, updatedAt
);
const Note = mongoose.model("Note", noteSchema); // creates a Note model based on noteSchema
export default Note;
```

Remember the **<db_name>** section in the mongodb link? This is where it appears:



Postman is one of the coolest things ever.

With postman we can do different http requests quickly and give input and receive a response. It is extremely useful for testing our API and the correctness of it.

Take the below code as an example:

```
export async function createNote(req, res){
  try{
    const {title, content} = req.body; ← The body of the request message we send to the
server
    const note = new Note({title:title, content:content}); ← Make a new note using the
request's body

    const savedNote = await note.save(); ← the actual part that sends the insert command
    res.status(201).json(savedNote); ← return HTTP 201 (updated) and show the saved note)
  } catch(error){
    console.error("Error in createNote controller", error);
    res.status(500).json({message:"Internal server error"});
  }
}
```

To be able to read “**req**” data (the data we send to the server in the body), we add this line to **server.js**:

```
connectDB(); ← was already there. Ignore this.

// middleware
app.use(express.json()); // lets us use the req body (the data we send to the server) in our code.
(In the controllers)
// Otherwise "req" would be null all the time.

app.use("/api/notes", notesRoutes); ← was already there. Ignore this.
```

Middleware

Middleware is a function that sits between the incoming request and your final route handler. Middleware runs on the way IN, before route handler processes the request.

`app.use();` is used to add a middleware.

Examples

```
app.use(express.json());
```

- This is a built-in Express middleware that parses incoming request bodies. When your frontend sends a POST/PUT request with JSON data (like `{title: "...", content: "..."}`), it arrives as a raw string/buffer — this middleware reads that, parses it into an actual JS object, and attaches it to `req.body`

```
app.use((req, res, next) => {  
  console.log(`New Request: method: ${req.method} URL: ${req.url}`);  
  next();  
})
```

- This is a custom middleware. Every single request that hits the server — no matter which route — passes through here first, and it logs the HTTP method (GET, POST, etc.) and the URL that was hit.

```
app.use("/api/notes", notesRoutes);
```

- This one's slightly different — it's not a generic "run on every request" middleware, it's **mounting the router** at a specific path prefix. It means: any request whose URL starts with **“/api/notes”**, hand it off to the **notesRoutes** router to figure out the rest.

Middleware is extremely useful since it allows us to implement features like authentication, rate limits (to avoid spam requests, DOS/DDOS attacks, etc.) and so on.

ERROR 429: Too Many Requests.

Upstash

For rate limiting we can use Upstash.

- Login on their website
- Make a database
- Put these values you find on the upstage database page in the env file:
 - UPSTASH_REDIS_REST_URL=https://cool-fj3something.upstash.io
 - UPSTASH_REDIS_REST_TOKEN=SomeLongTokenHere
- In the terminal run: **npm install @upstash/ratelimit@2.0.5 @upstash/redis@1.34.9**
- Make **src/config/upstash.js**

```
import {Ratelimit} from "@upstash/ratelimit";
import {Redis} from "@upstash/redis";
import dotenv from "dotenv";
dotenv.config();
// create a ratelimiter that allows 10 requests per 20 seconds
const ratelimit = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(100, "60 s"), ← 5 requests every 10 seconds
});
export default ratelimit;
```

- Make **src/middleware/rateLimiter.js**

```
import ratelimit from "../config/upstash.js";
const rateLimiter = async (req, res, next) => {
  try{
    const {success} = await ratelimit.limit("my-limit-key");
    if (!success){
      return res.status(429).json({message:"Too many requests, please try again later."});
    }
    next();
  } catch(error){
    console.log("Rate limit error", error);
    next(error);
  }
}
export default rateLimiter;
```

Front-end

Setting up the frontend:

- Open a terminal and head to **./frontend/**
- Run **npm create vite@latest .**
- Choose: **React, JavaScript**
- Run **npm install react-router** ← Used for switching between webpages
- Run **npm install react-hot-toast** ← Used for popup notifications (succes, fail, etc.)

In **./frontend/src/**

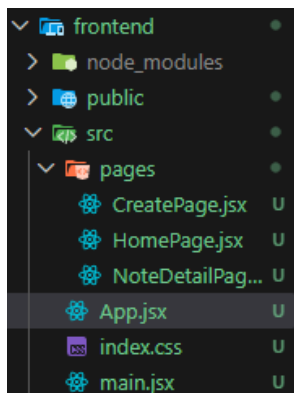
- Delete **./frontend/src/assets/**
- Delete **App.css**
- Clear **App.jsx**
- Clear **index.css**

We start by making these files:

```
main.jsx x App.jsx x tailwind.config.js x
frontend > src > main.jsx
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.jsx'
5 import { BrowserRouter } from "react-router";
6 import { Toaster } from "react-hot-toast";
7
8 createRoot(document.getElementById('root')).render(
9   <StrictMode>
10     <BrowserRouter>
11       <App />
12       <Toaster />
13     </BrowserRouter>
14   </StrictMode>,
15 )
16
17
```

```
main.jsx x App.jsx x tailwind.config.js x
frontend > src > App.jsx > ...
1 import HomePage from './pages/HomePage';
2 import CreatePage from './pages/CreatePage';
3 import NoteDetailPage from './pages/NoteDetailPage';
4 import { Route, Routes } from 'react-router';
5 import toast from 'react-hot-toast';
6
7 const App = () => {
8   return (
9     <div>
10       <button onClick={ () => toast.success("congrats")}>success test</button>
11       <button onClick={ () => toast.error("failed")}>error test</button>
12       <Routes>
13         <Route path="/" element={<HomePage/>} />
14         <Route path="/create" element={<CreatePage/>} />
15         <Route path="/note/:id" element={<NoteDetailPage/>} />
16       </Routes>
17     </div>
18   )
19 }
20
21 export default App
22
```

<Toaster/> in **main.jsx** is like a layer on top of everything (since main.jsx is our root file) and it allows the child layers (like App.jsx) to access the **toast** notification messages and use its methods.



And we make a folder holding our pages (which we are going to develop later on).

We use a **<Router>** tag with **<Route>** tags inside of it for different routes on our website. This is how we **connect our pages together**.

tailwindcss

We have to install tailwindcss since it allows us to use css inside of our jsx files instead of having to make a separate css file for each.

Open terminal at ./backend/

- Run **npm install -D tailwindcss@latest @tailwindcss/vite**
- Then put these in **vite.config.js**:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

- For the final step, add these to **index.css**:

```
@import "tailwindcss";
```

daisyUI

Now we install DaisyUI for shorter code, better looks, and faster UI.

- Run **npm install -D daisyui@latest** at ./frontend/
- Change the tailwind.config.js to:

```
import daisyui from "daisyui";
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [daisyui],
}
```

To select a specific theme:

- Go to <https://daisyui.com/docs/themes/> and choose a theme you would like to use.
- Add then add this (+ the theme you want) to **index.css**:

```
@plugin "daisyui" {
  themes: abyss --default, dark, light; ← The themes you want to use (LOOKUP from daisyui.com)
}
```

Note: the chosen theme becomes the whole frontend's theme. You can change different parts of each page by just adding **data-theme="THEME_NAME"** to that section's tag.

Icons

Since we will use many different icons, it is better if we install a package that has them and use those instead.

At `./frontend/` :

- run: **npm install lucide-react**
- run: **npm install axios** ← To replace our fetch API with axios (for later)

Navbar.jsx

It's a best practice to make a components folder for footer, header, navbar, etc.

```
frontend > src > components > 🗂️ Navbar.jsx > ...
1  import { Link } from "react-router";
2  import { PlusIcon } from "lucide-react";
3
4  const Navbar = () => {
5    return (
6      <header className="bg-base-300 border-b border-base-content/10">
7        <div className="mx-auto max-w-6xl px-5 py-4"> { /* px and py are padding x and y (p-4 for all sides) */}
8          <div className="flex items-center justify-between">
9            <h1 className="text-3xl font-bold text-primary font-mono tracking-tighter">ToDo list</h1>
10           <div className="flex items-center gap-4">
11             <Link to={"/create"} className="btn btn-primary">
12               <PlusIcon className="h-5 w-5"/> { /* hight 5 and width 5 can be also said as: size-5 */}
13               <span>New Note</span>
14             </Link>
15           </div>
16         </div>
17       </div>
18     </header>
19   )
20 }
21
```

Notice how **Link** is imported from **react-router** and how `<Link to={"/create"}>` is used to make a link to our other **route** (webpage).

RateLimitedUI.jsx

A component to show the user they have hit the 429 (too many requests) error for spamming.

```
frontend > src > components > ⚙️ RateLimitedUI.jsx > ...
1  import { ZapIcon } from "lucide-react";
2
3  const RateLimitedUI = () => {
4    return (
5      <div className="max-w-6xl mx-auto px-4 py-8">
6        <div className="bg-primary/10 border border-primary/30 rounded-lg shadow-md">
7          <div className="flex flex-col md:flex-row items-center p-6">
8            <div className="flex-shrink-0 bg-primary/20 p-4 rounded-full mb-4 md:mb-0 md:mr-6">
9              <ZapIcon className="size-10 text-primary" />
10             </div>
11             <div className="flex-1 text-center md:text-left">
12               <h3 className="text-xl font-bold mb-2">Rate Limit Reached</h3>
13               <p className="text-base-content mb-1">
14                 You've made too many requests in a short period. Please wait a moment.
15               </p>
16               <p className="text-sm text-base-content/70">
17                 Try again in a few seconds for the best experience.
18               </p>
19             </div>
20           </div>
21         </div>
22       </div>
23     );
24   };
25
26   export default RateLimitedUI;
```

Cross-Origin Resource Sharing (CORS)

When a website tries to get data from another website - like your frontend calling an API on a different domain- the browser might block it for security reasons.

Example:

- Frontend at: <http://localhost:3000>
- API backend at: <http://api.example.com>

To fix it:

- Go to **./backend/** in terminal and run **npm install cors**
- Add these to server.js (backend) :

```
import cors from "cors";
app.use(cors({
  origin: "http://localhost:5173", ← The address of the API (can have multiple in array)
}));
```

NOTE: app.use(cors()); must come before every other middleware.

Homepage.jsx

```

frontend > src > pages > HomePage.jsx > ...
1  import { useEffect, useState } from "react";
2  import Navbar from "../components/Navbar";
3  import RateLimitedUI from "../components/RateLimitedUI";
4  import axios from "axios";
5  import toast from "react-hot-toast";
6  import NoteCard from "../components/NoteCard";
7
8
9  const HomePage = () => {
10     const [isRateLimited, setIsRateLimited] = useState(false);
11     const [notes, setNotes] = useState([]);
12     const [loading, setLoading] = useState(true); /* true since we will try to fetch notes as soon as we get on the homepage*/
13
14     useEffect(() => {
15         const fetchNotes = async () => {
16             try {
17                 const res = await axios.get("http://localhost:5001/api/notes");
18                 console.log(res.data);
19                 setNotes(res.data);
20                 setIsRateLimited(false);
21             } catch (error) {
22                 console.log("Error fetching notes");
23                 // console.log(error);
24                 if (error.response?.status == 429) {
25                     setIsRateLimited(true);
26                 } else {
27                     toast.error("Failed to load notes");
28                 }
29             } finally {
30                 setLoading(false);
31             }
32         }
33         fetchNotes();
34     }, []);
35
36     return (
37         <div className="min-h-screen">
38             <Navbar />
39             {isRateLimited && <RateLimitedUI />} /* if isRateLimited, return <RateLimitedUI /> */
40             <div className="max-w-7xl mx-auto px-4 mt-6">
41                 {loading && <div className="text-center text-primary text-5xl py-10">Loading notes...</div>} /* if loading, show the loading notes message */
42                 {notes.length > 0 && !isRateLimited && (
43                     <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
44                         {notes.map(note => (
45                             <NoteCard key={note._id} note={note} />
46                         ))}
47                     </div>
48                 )}
49             </div>
50         </div>
51     );
52 }
53
54 export default HomePage;

```

Instead of using `axios.get("http://localhost:5001/api/notes");` we can use this other method:

```

frontend > src > lib > axios.js > ...
1  import axios from "axios";
2  const api = axios.create({
3      |   baseURL: "http://localhost:5001/api/"
4  | })
5  export default api;

```

And then do:

```

import api from "../lib/axios";
...
... api.get("/notes");

```

CreatePage.jsx

```

frontend > src > pages > CreatePage.jsx > _
1  import axios from "axios";
2  import { ArrowLeftIcon } from "lucide-react";
3  import { useState } from "react";
4  import toast from "react-hot-toast";
5  import { Link, useNavigate } from "react-router";
6
7
8  const CreatePage = () => {
9    const [title, setTitle] = useState("");
10   const [content, setContent] = useState("");
11   const [loading, setLoading] = useState(false);
12   const navigate = useNavigate();
13
14   const handleSubmit = async (e) => {
15     e.preventDefault(); // prevent refresh each time button is pressed
16     // if (!title.trim() || !content.trim()){
17     //   toast.error("All fields are required.");
18     //   return;
19     // }
20     setLoading(true);
21     try{
22       await axios.post("http://localhost:5001/api/notes", {
23         title,
24         content
25       });
26       toast.success("Note created.");
27       navigate("/");
28     } catch (error) {
29       console.log("Error creating note", error);
30       if (error.response.status == 429){
31         toast.error("Slow down! You're creating notes too fast", {
32           duration: 4000,
33           icon: "🧠",
34         });
35       } else{
36         toast.error("Failed to create note.");
37       }
38     } finally {
39       setLoading(false);
40     }
41   }
42
43   return (
44     <div className="min-h-screen bg-base-200">
45       <div className="container mx-auto px-4 py-8">
46         <div className="max-w-2xl mx-auto">
47           <Link to="/" className="btn btn-ghost mb-6 text-primary">
48             <ArrowLeftIcon className="size-5" />
49             Back to Notes
50           </Link>
51
52           <div className="card bg-base-100">
53             <div className="card-body">
54               <h2 className="card-title text-2xl mb-4 text-primary">Create New Note</h2>
55               <form onSubmit={handleSubmit}>
56
57                 <fieldset className="form-control mb-4">
58                   <legend className="label-text text-xl px-5 py-2 text-primary">Title</legend>
59                   <input
60                     type="text"
61                     placeholder="Note Title"
62                     className="input input-bordered w-full rounded-full"
63                     value={title}
64                     onChange={(e) => setTitle(e.target.value)} />
65                 </fieldset>
66
67                 <fieldset className="form-control mb-4">
68                   <legend className="label-text text-xl px-5 py-2 text-primary">Content</legend>
69                   <textarea
70                     placeholder="Write your note here..."
71                     className="textarea textarea-bordered h-32 w-full"
72                     value={content}
73                     onChange={(e) => setContent(e.target.value)} />
74                 </fieldset>
75
76                 <div className="card-actions justify-end">
77                   <button type="submit" className="btn btn-primary disabled={loading}>
78                     {loading ? "Creating..." : "Create Note"}
79                   </button>
80                 </div>
81               </div>
82             </div>
83           </div>
84         </div>
85       </div>
86     </div>
87   )
88
89 </div>
90
91 }
92
93 export default CreatePage;

```